Numerical Linear Algebra for Computational Science and Information Engineering

Lecture 17
Introduction to Communication-Avoiding Algorithms

Nicolas Venkovic
nicolas.venkovic@tum.de

Group of Computational Mathematics
School of Computation, Information and Technology
Technical University of Munich

Summer 2025

Outline

1. Introduction
   CS294/Math270 – Demmel and Grigori (2016)
   MIT 6.172 – Shun (2018) 1
2. Matrix-matrix multiplication
   MIT 6.172 – Shun (2018) 7
3. Overview and principles of communication avoidance 15
4. Sparse matrix-vector product (SpMV) 17
5. Block Gram-Schmidt procedures 23
6. s-step iterative solvers 23
7. Matrix power kernels 23
8. Homework problems 23
9. Practice session 23

Introduction

CS294/Math270 – Demmel and Grigori (2016)
MIT 6.172 – Shun (2018)

Cost of algorithm deployment

- Deploying an algorithm has **two costs** measured in **time** (or **energy**):
  - **Arithmetic** (floating-point operations, FLOPs)

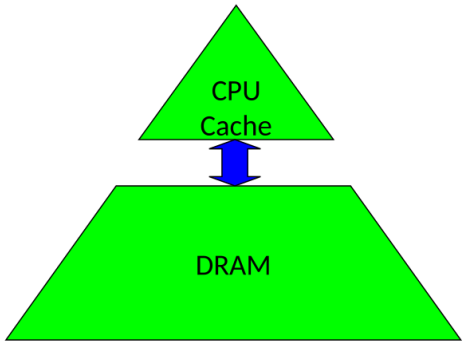
$$\boxed{\#\text{ of }\textbf{FLOPs} \div (\#\text{ of }\textbf{FLOPs per cycle} \times \#\textbf{ cycles per unit of time})}$$

  - **Communication** due to data movement between
    - **levels of a memory hierarchy**:

      sequential part of a program

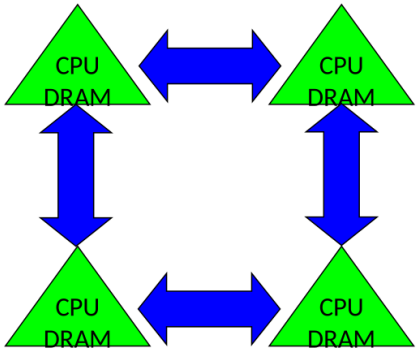
      transfers between DRAM, L3 cache, L2 cache, L1 cache and registers

      cache hit, cache miss, ...



    - **processors over a network**:

      parallel part of a program



$$\boxed{\#\text{ of }\textbf{messages} \times \textbf{latency} + \#\text{ of }\textbf{words} \div \textbf{bandwidth}}$$

$$\boxed{\textbf{idle}\text{ time}}$$

Roofline model

- **Algorithm** on given **hardware** characterized by **arithmetic intensity**:

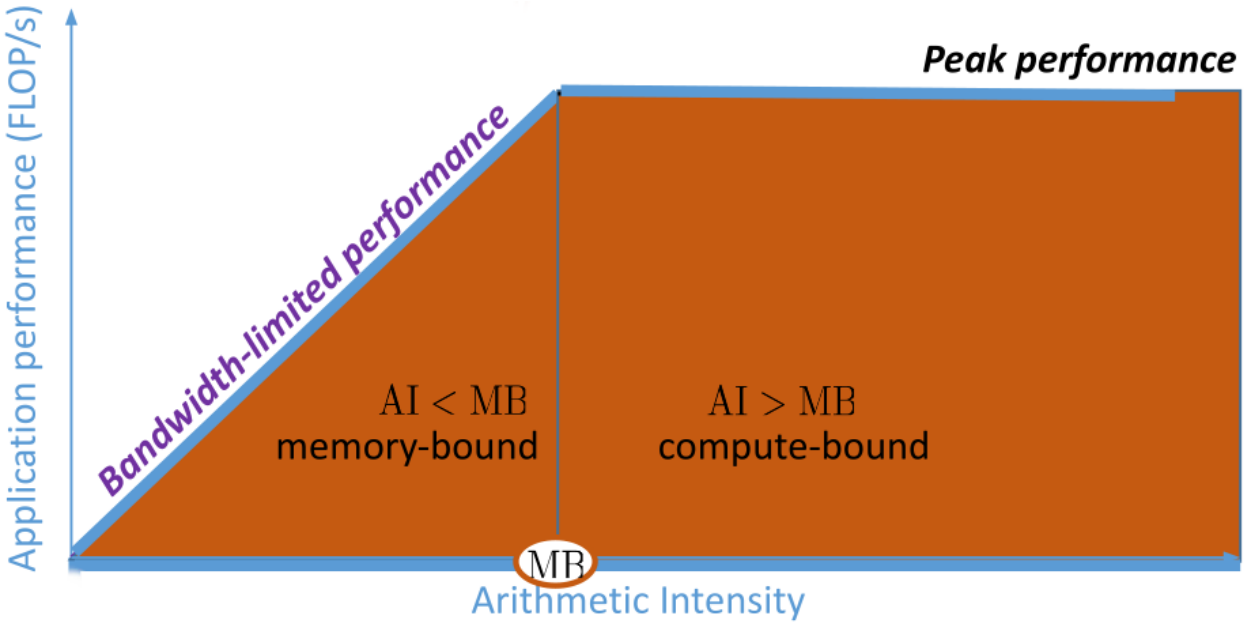
Arithmetic intensity (AI)

Ratio of number of FLOPs over amount of data moved.

- **Hardware** characterized by **machine balance**:

Machine balance (MB)

Ratio of peak floating-point performance over peak memory bandwidth.

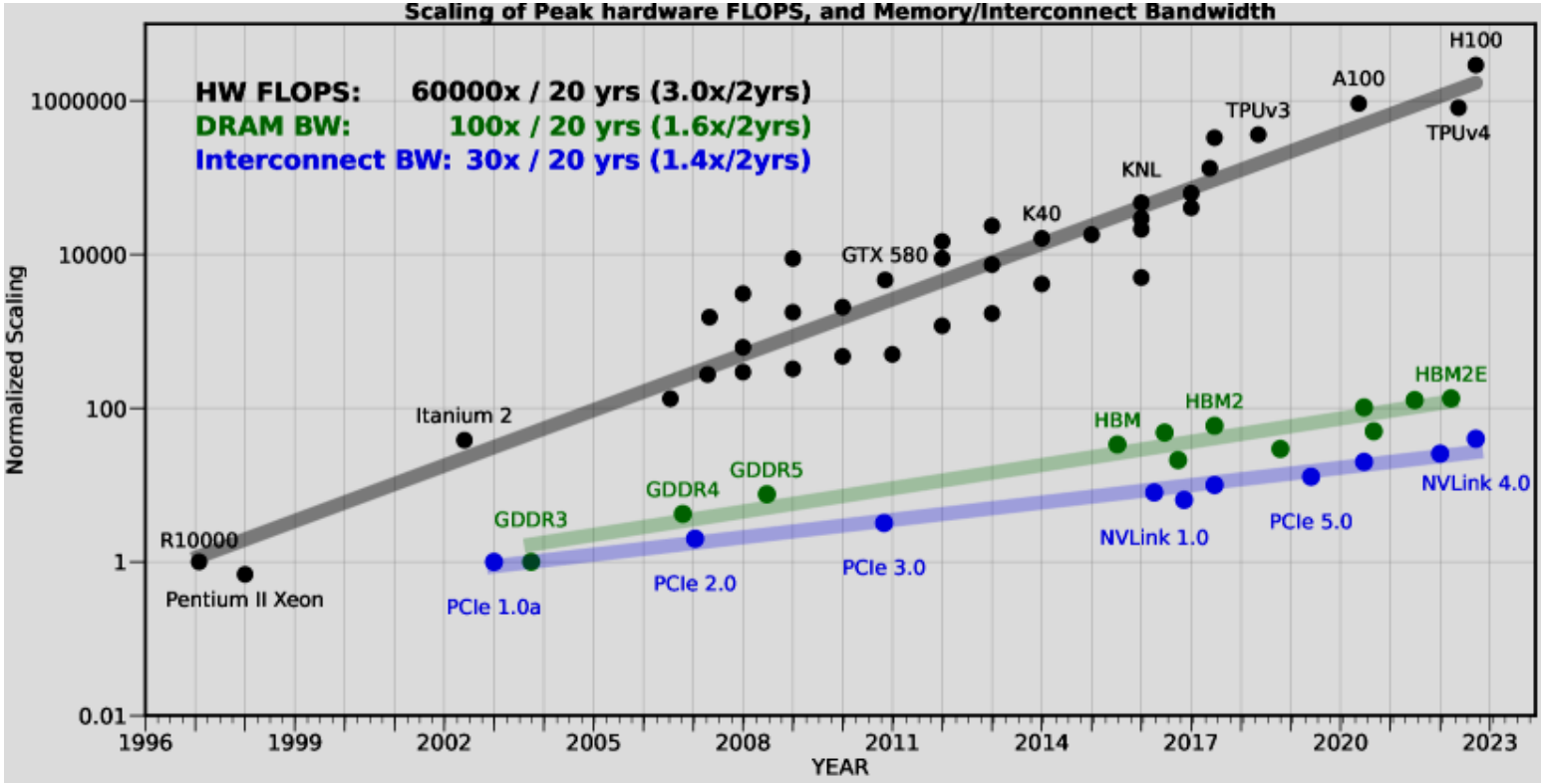


Memory wall

- Cost to move data much greater than cost of arithmetic:

$$\textbf{time per FLOP} \ll \frac{1}{\textbf{bandwidth}} \ll \textbf{latency}$$

- Peak floating-point performance evolves faster than memory bandwidth:

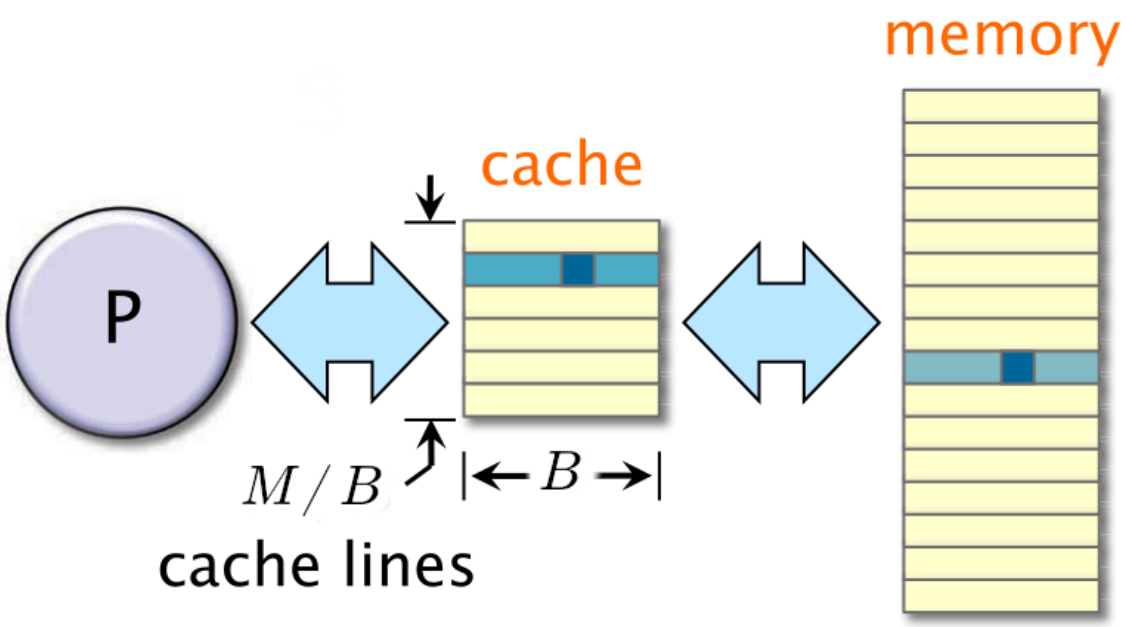


- Relative cost of algorithms due to communication larger every year

Gholami, A., Yao, Z., Kim, S., Hooper, C., Mahoney, M.W., Keutzer, K. (2024). AI and Memory Wall. arXiv:2403.14123v1.

Ideal cache model

- An **ideal cache model** is introduced for the analysis of algorithms:



- Model parameters:
  - Two-level hierarchy
  - Cache size of M bytes
  - Cache line length of B bytes
  - Fully associative cache (cache lines can be stored anywhere in cache)
  - Least-recently used (LRU) cache replacement policy
  - Arbitrary large main memory

Tall caches

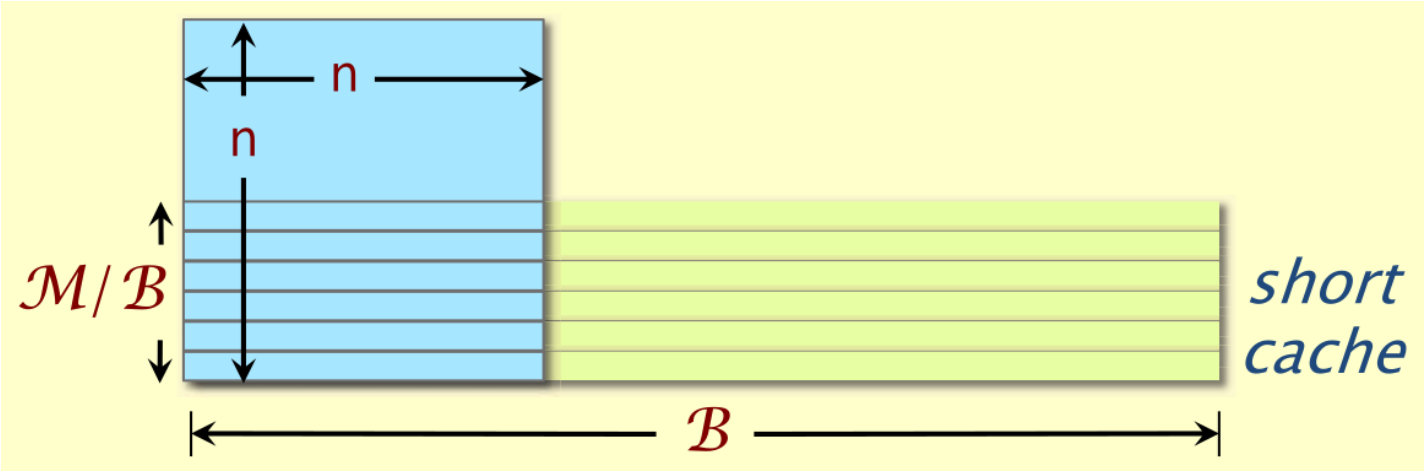
- Several of the coming cache analyses assume a tall cache:

Tall cache assumption

A cache of size M bytes with N/B lines of B bytes is **tall** if $B^2 < cM$ for some sufficiently small $c < 1$.

Problem of **short cache**:

- a $n \times n$ **sub**matrix may not fit in cache, even if $n^2 < cM$



i.e., cache lines store only contiguous data, and consecutive rows of a row major submatrix are not contiguous in memory.

Communication lower bounds

- Assuming a fast memory of size M, Ballard et al. (2011) derive **lower bounds** on communication for valid all methods of **direct linear algebra**, **dense** or **sparse**:

$$\textbf{bandwidth cost}: \ \# \text{ of words moved} = \Omega\left(\frac{\# \text{ of arithmetic operations}}{\sqrt{M}}\right)$$

$$\textbf{latency cost}: \ \# \text{ of messages sent} = \Omega\left(\frac{\# \text{ of arithmetic operations}}{\sqrt{M^3}}\right)$$

- These bounds hold for
  - Matrix-multiply, LU, QR, eigensolving, SVD, tensor contraction, ...
  - Some whole programs (sequences of these operations, no matter how individual operations are interleaved, e.g. A^k)
  - **Sequential** and **parallel** algorithms
  - Some graph-theoretic algorithms (e.g. Floyd-Warshall)
- Significant efforts by the NLA community to rethink algorithms to try and achieve these bounds.

Ballard, G., Demmel, J., Holtz, O., & Schwartz, O. (2011). Minimizing communication in numerical linear algebra. SIAM Journal on Matrix Analysis and Applications, 32(3), 866-901.

Matrix-matrix multiplication

MIT 6.172 – Shun (2018)

Cache misses analysis

- Matrix-matrix multiplication with row major data:

```
for i = 0, ..., n-1                                  //zero-based indexing
  for j = 0, ..., n-1
    C[i*n+j] := 0
    for k = 0, ..., n-1
      C[i*n+j] := C[i*n+j] + A[i*n+k] * B[k*n+j]
```

Cache miss complexity, $Q(n)$, **dominated by access to** B.

Assume a **tall** and **ideal cache**. There are 3 cases:

- Case 1: $n > cM/B$

  B misses on every access $\implies Q(n) = \Theta(n^3)$
- Case 2: $c'M^{1/2} < n < cM/B$

  B exploits spatial locality $\implies Q(n) = \Theta(n^3/B)$
- Case 3: $n < cM^{1/2}$

  B fits in cache $\implies Q(n) = \Theta(n^2/B)$

Blocked (tiled) matrix multiplication

- Introduce a block size s:

```
for i_1 = 0, s, ..., n − s                                  //zero-based indexing
  for j_1 = 0, s, ..., n − s
    for k_1 = 0, s, ..., n − s
      for i = i_1, ..., i_1 + s − 1
        for j = j_1, ..., j_1 + s − 1
          C[i * n + j] := 0
          for k = k_1, ..., k_1 + s − 1
            C[i * n + j] := C[i * n + j] + A[i * n + k] * B[k * n + j]
```

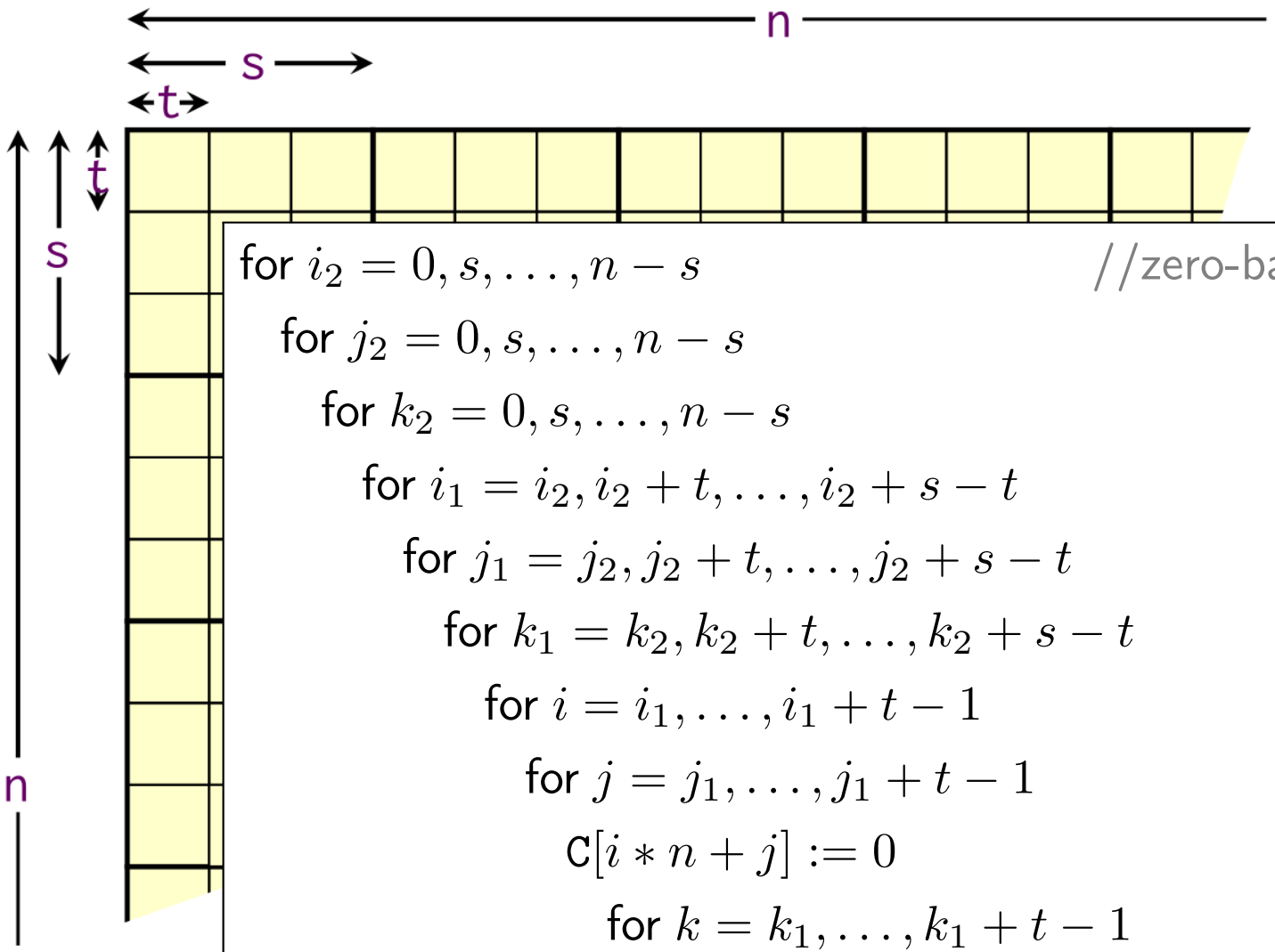
Tune s so that the submatrices fit in cache, i.e., $s = \Theta(M^{1/2})$.

$\implies$ **cache-aware** algorithm, **architecture dependent**.

Then $\Theta(s^2/B)$ misses per submatrix so that

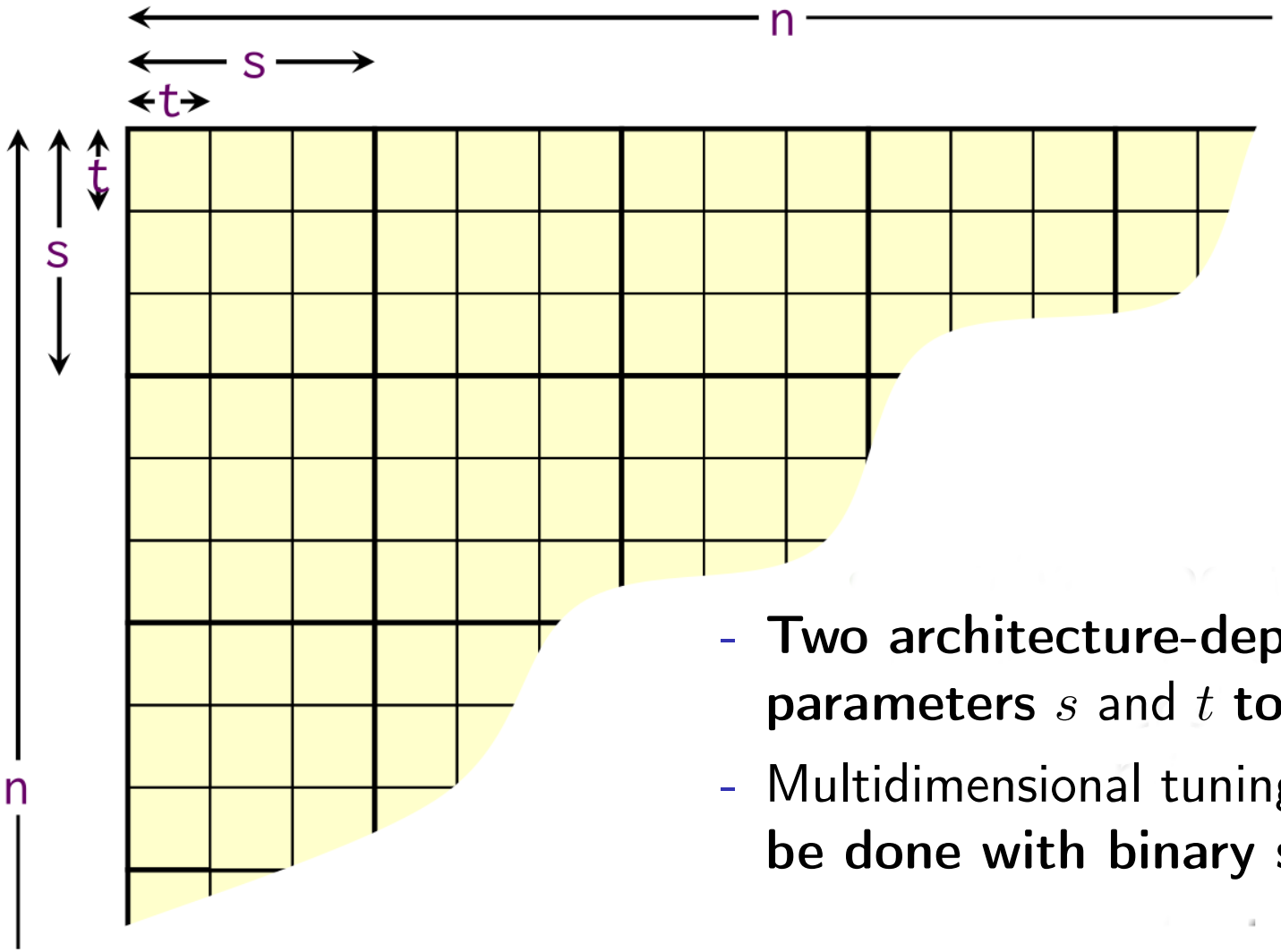
$$Q(n) = \Theta((n/s)^3(s^2/B)) = \Theta(n^3/(BM^{1/2})) \rightarrow \text{ provably } \textbf{optimal}.$$

Two-level cache



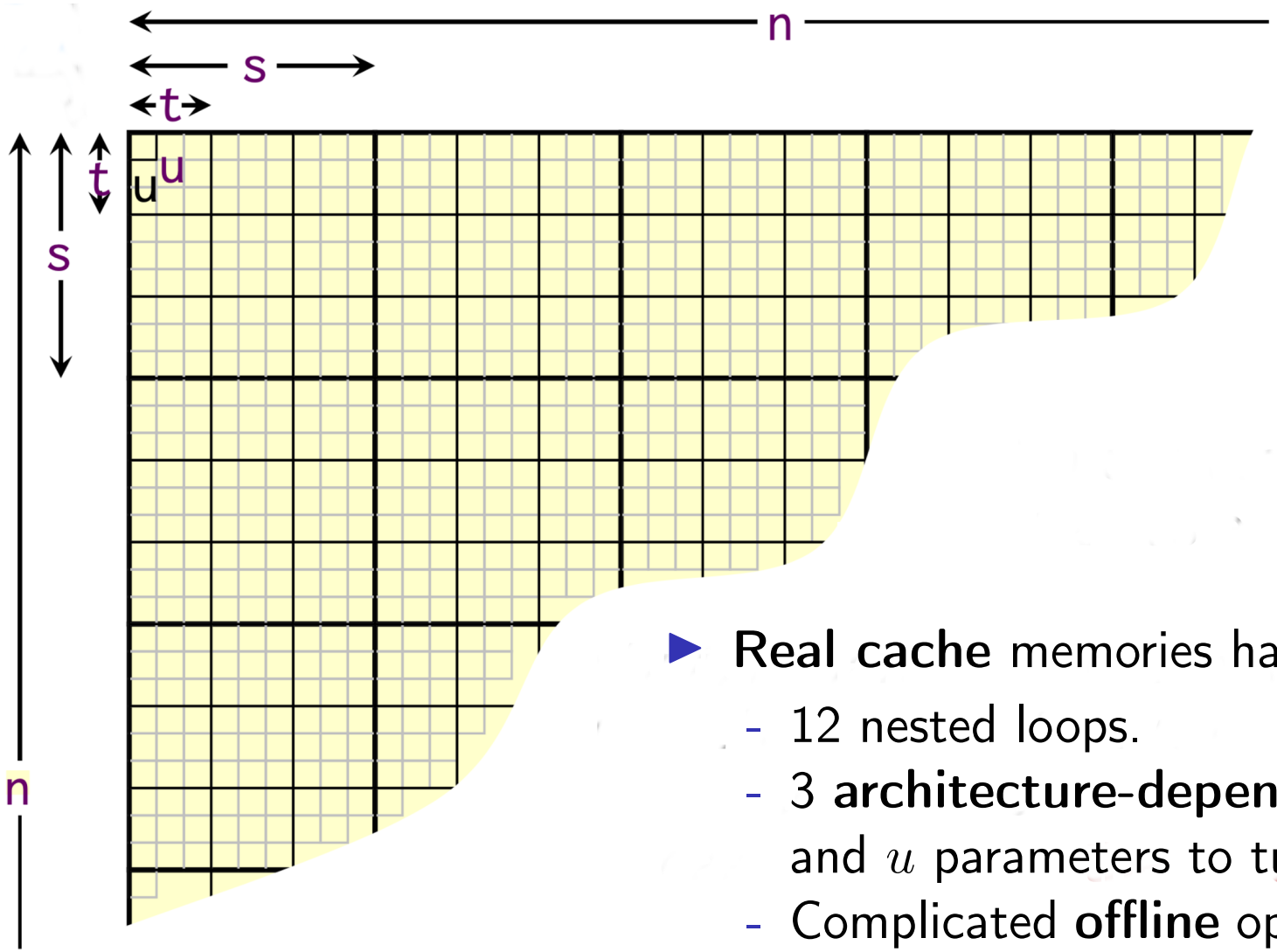

```
for i_2 = 0, s, ..., n − s                                  //zero-based indexing
  for j_2 = 0, s, ..., n − s
    for k_2 = 0, s, ..., n − s
      for i_1 = i_2, i_2 + t, ..., i_2 + s − t
        for j_1 = j_2, j_2 + t, ..., j_2 + s − t
          for k_1 = k_2, k_2 + t, ..., k_2 + s − t
            for i = i_1, ..., i_1 + t − 1
              for j = j_1, ..., j_1 + t − 1
                C[i * n + j] := 0
                  for k = k_1, ..., k_1 + t − 1
                    C[i * n + j] := C[i * n + j]
                                  + A[i * n + k] * B[k * n + j]
```

Two-level cache, cont'd



- **Two architecture-dependent parameters** s and t **to tune**.
- Multidimensional tuning **cannot be done with binary search**.

Three-level cache



- **Real cache** memories have 3 levels:
  - 12 nested loops.
  - 3 **architecture-dependent** s, t and u parameters to tune.
  - Complicated **offline** optimization.
- Existing implementations:
  - OpenBLAS, BLIS, ATLAS.

Recursive matrix multiplication

- Divide-and-conquer on $n \times n$ matrices:

$$\begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix} = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \times \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}$$

$$= \begin{bmatrix} A_{11}B_{11} & A_{11}B_{12} \\ A_{21}B_{11} & A_{21}B_{12} \end{bmatrix} + \begin{bmatrix} A_{12}B_{21} & A_{12}B_{22} \\ A_{22}B_{21} & A_{22}B_{22} \end{bmatrix}$$

Recurrence of 8 multiply-and-add of $n/2 \times n/2$ matrices.

There are two cases of memory access:

- Case 1: $n < cM^{1/2} \implies Q(n) = \Theta(n^2/B)$
- Case 2: otherwise $\implies Q(n) = 8Q(n/2) + \Theta(1)$

  (Master theorem) $\implies Q(n) = \Theta(n^3/(BM^{1/2}))$

Recursive implementation

- Assuming n is a power of 2:

$\texttt{recursive_gemm}(A, B, C, n):$ $\quad //C_{1:n,1:n} = 0$

if $(n = 1):$

$c_{11} := c_{11} + a_{11} * b_{11}$ $\quad$ //one-based indexing

else :

$\texttt{recursive_gemm}(A_{1:\frac{n}{2},1:\frac{n}{2}}, B_{1:\frac{n}{2},1:\frac{n}{2}}, C_{1:\frac{n}{2},1:\frac{n}{2}}, n/2)$

$\texttt{recursive_gemm}(A_{1:\frac{n}{2},\frac{n}{2}+1:n}, B_{\frac{n}{2}+1:n,1:\frac{n}{2}}, C_{1:\frac{n}{2},\frac{n}{2}+1:n}, n/2)$

$\texttt{recursive_gemm}(A_{1:\frac{n}{2},1:\frac{n}{2}}, B_{1:\frac{n}{2},\frac{n}{2}+1:n}, C_{1:\frac{n}{2},\frac{n}{2}+1:n}, n/2)$

$\texttt{recursive_gemm}(A_{1:\frac{n}{2},\frac{n}{2}+1:n}, B_{\frac{n}{2}+1:n,\frac{n}{2}+1:n}, C_{1:\frac{n}{2},\frac{n}{2}+1:n}, n/2)$

$\texttt{recursive_gemm}(A_{\frac{n}{2}+1:n,1:\frac{n}{2}}, B_{1:\frac{n}{2},1:\frac{n}{2}}, C_{\frac{n}{2}+1:n,1:\frac{n}{2}}, n/2)$

$\texttt{recursive_gemm}(A_{\frac{n}{2}+1:n,\frac{n}{2}+1:n}, B_{\frac{n}{2}+1:n,1:\frac{n}{2}}, C_{\frac{n}{2}+1:n,1:\frac{n}{2}}, n/2)$

$\texttt{recursive_gemm}(A_{\frac{n}{2}+1:n,1:\frac{n}{2}}, B_{1:\frac{n}{2}:n,\frac{n}{2}+1:n}, C_{\frac{n}{2}+1:n,\frac{n}{2}+1:n}, n/2)$

$\texttt{recursive_gemm}(A_{\frac{n}{2}+1:n,\frac{n}{2}+1:n}, B_{\frac{n}{2}+1:n,\frac{n}{2}+1:n}, C_{\frac{n}{2}+1:n,\frac{n}{2}+1:n}, n/2)$

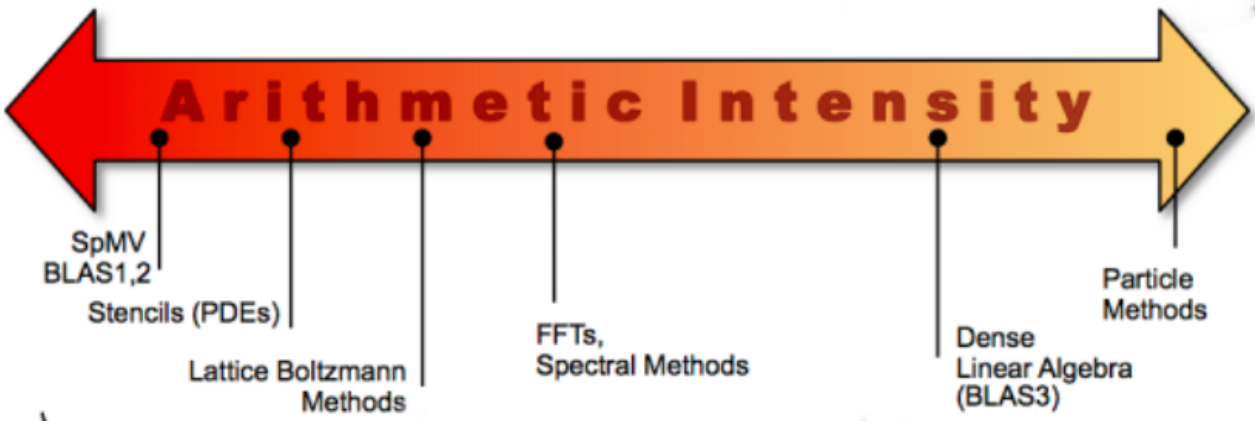
Efficient cache-oblivious algorithms

- **Cache complexity** of `recursive_gemm` **similar to** that of **tiled matrix multiply**.
- Implementation of `recursive_gemm` **not dependent on memory layout**. It is a **cache-oblivious** algorithm.
- The best cache-oblivious algorithm work on arbitrary rectangular matrices and perform binary splitting (instead of 8-way).
- Recursive matrix-multiply is **typically not implemented in BLAS**, **despite good theoretical cache behavior**, because:
  - It incurs overhead from recursion and function calls.
  - Difficult to efficiently vectorize and parallelize on modern hardware.
  - Hard to outperform hand-optimized, architecture-aware, cache-aware implementations.
- Recursive matrix multiply is more used as a pedagogical tool for illustrating cache efficiency and divide-and-conquer design.
- Recursion is the basis of other fast matrix multiply algorithms, e.g., Strassen's method.

Overview and principles of communication avoidance

Reducing data movement

- Principles to reduce communication:
  - **Maximize arithmetic intensity**: Perform more floating-point operations per byte of data moved.



  - **Exploit data locality**: Reuse data in fast memory (cache/registers) as much as possible.
  - **Aggregate communication**: Communicate in bulk instead of fine-grained messages.
- Associated challenges:
  - **Numerical stability**: Communication-avoiding variants may be less stable or harder to analyze.
  - **Architecture-specific tuning**: Optimal strategies depend on cache hierarchy, network topology, etc...

Sample speedups

- Demmel and Grigori (2016) documents the following speedups achieved by reformulating NLA and other kernels to reduce communication:
  - Up to **12X** faster for **2.5D matrix multiply** on 64K core IBM BG/P
  - Up to **3X** faster for **tensor contractions** on 2K core Cray XE/6
  - Up to **6.2X** faster for **All-Pairs-Shortest-Path** on 24K core Cray CE6
  - Up to **2.1X** faster for **2.5D LU** on 64K core IBM BG/P
  - Up to **11.8X** faster for **direct N-body** on 32K core IBM BG/P
  - Up to **13X** faster for **Tall Skinny QR** on Tesla C2050 Fermi NVIDIA GPU
  - Up to **6.7X** faster for **symeig(band A)** on 10 core Intel Westmere
  - Up to **2X** faster for **2.5D Strassen** on 38K core Cray XT4
  - Up to **4.2X** faster for MiniGMG benchmark bottom solver, using **CA-BiCGStab** (**2.5X** for overall solve)
  - Up to **2.5X** for combustion simulation code

Demmel J. & Grigori L. (2016) Introduction to communication-avoiding algorithms. CS294/Math270, UC Berkeley.

Sparse matrix-vector product (SpMV)

Low data locality of SpMV

- Recall the CSR data structure:

$$A = \begin{bmatrix} a_{11} & a_{12} & a_{13} & 0 \\ a_{21} & a_{22} & 0 & 0 \\ 0 & 0 & a_{33} & a_{34} \\ 0 & 0 & a_{43} & 0 \end{bmatrix}$$

$$\texttt{val} = [a_{11}, a_{12}, a_{13}, a_{21}, a_{22}, a_{33}, a_{34}, a_{43}]$$

$$\texttt{col_idx} = [1, 2, 3, 1, 2, 3, 4, 3]$$

$$\texttt{row_start} = [1, 4, 6, 8, 9]$$

with the following SpMV kernel:

```
for i = 1, ..., n                                  //one-based indexing
  sum := 0
  for j = row_start[i], ..., row_start[i + 1] − 1
    sum := sum + val[j] ∗ x[col_idx[j]]
  y[i] := sum
```

Low data locality of SpMV, cont'd

- SpMV kernel for CSR data structures:

```
for i = 1, ..., n                                  //one-based indexing
  sum := 0
  for j = row_start[i], ..., row_start[i + 1] − 1
    sum := sum + val[j] ∗ x[col_idx[j]]
  y[i] := sum
```

- **Irregular access** to the components of x:
  - **No spatial locality**, **no temporal locality**.
  - **Every component** of x loaded for a single multiply-and-add is **trashed immediatly from register**, i.e., no loop unrolling, no SIMD, ...
  - **Sparsity induces lower arithmetic intensity** (AI) than GEMV.
  - **SpMV kernels** are **memory-bound**, even if x fits in cache.
  - **Performance** of SpMV kernels depends on **data structure**, **non-zero structure** and **hardware**.

Register blocking

- Improve data locality by promoting **register reuse**:
  - **Register reuse** is **achieved** for the source vector **by blocking**, also known as **tiling**, i.e., relying on contiguous storage of non-zero blocks in the matrix.
  - **Registers** are small, i.e., only **store a handfull of data**:

    $\Longrightarrow$ small $r \times c$ block sizes : r values of destination vector + c values of source vector + rc values of a block are stored in register.
  - Blocking allows for **compiler optimizations**, e.g., loop unrolling, pipelining, ...
  - **Optimal block size dependent** on **sparsity strucutre** and **machine architecture**.
  - Considerable **overhead** in changing data structure for register blocking.
  - Applications such as finite element methods (FEM) naturally store small dense blocks in sparse matrices.
  - Most matrices do not have uniform block structures:

    $\Longrightarrow$ **some zero values** are **stored** for non-fully dense blocks.

Blocked SpMV

- Recall the BSR (or BCSR) data structure format:

$$A = \begin{bmatrix} a_{11} & a_{12} & a_{13} & 0 \\ a_{21} & a_{22} & 0 & 0 \\ 0 & 0 & a_{33} & a_{34} \\ 0 & 0 & a_{43} & 0 \end{bmatrix}$$

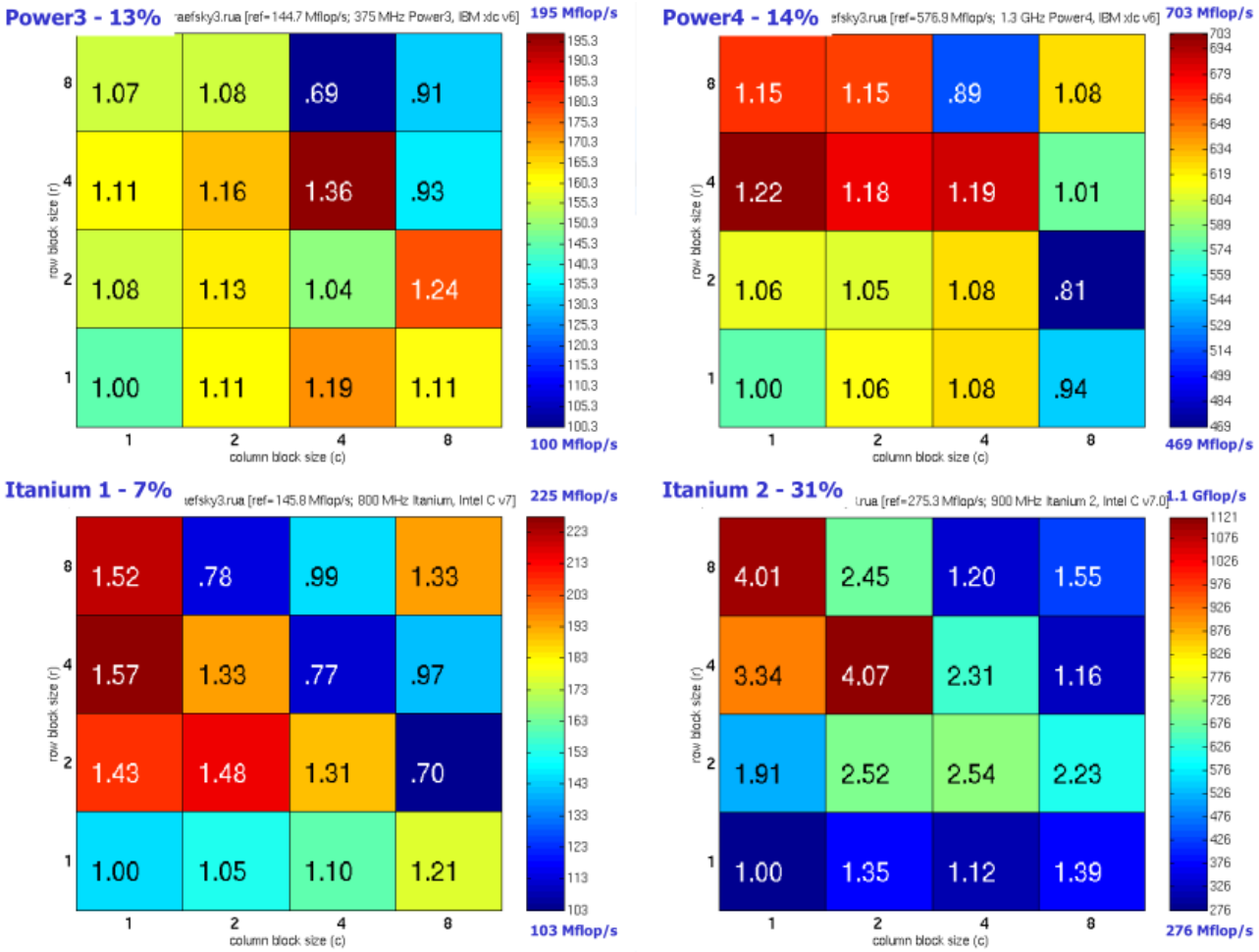
$$\begin{aligned} \texttt{r} = 2,\ \texttt{c} = 2 \\ \texttt{val} = [a_{11}, a_{12}, a_{21}, a_{22}, a_{13}, 0, 0, 0, \\ a_{33}, a_{34}, a_{43}, 0] \\ \texttt{col_idx} = [1, 2, 2],\ \texttt{row_start} = [1, 3, 4] \end{aligned}$$

with the following SpMV kernel:

```
for i = 1, ..., n/r                                          //one-based indexing
  for j = row_start[i], ..., row_start[i+1] - 1
    for ii = 1, ..., r
      sum := 0
      for jj = 1, ..., c
        sum := sum +
         val[(j-1)*rc + (ii-1)*c + jj] * x[(col_idx[j] - 1)*rc + jj]
      y[(i-1)*r + ii] := sum
```

Tuning of register blocks

- Optimal block size for register reuse depends on **sparsity** and **machine architecture**:
  - Automated tuning interface by Vuduc et al. (2005): OSKI



Vuduc, R., Demmel, J. W., & Yelick, K. A. (2005). OSKI: A library of automatically tuned sparse matrix kernels. In Journal of Physics: Conference Series (Vol. 16, No. 1, p. 521). IOP Publishing.

Encoding block sparsity

- **Zeros stored** in non-fully dense blocks **result in** extra floating-point operations, and more importantly, **extra data movement**.
- Only non-zero values need be stored and fetched when **local block sparsity** is **encoded**:

$$A = \begin{bmatrix} a_{11} & a_{12} & a_{13} & 0 \\ a_{21} & a_{22} & 0 & 0 \\ 0 & 0 & a_{33} & a_{34} \\ 0 & 0 & a_{43} & 0 \end{bmatrix} \qquad \begin{aligned} \texttt{r} = 2, \texttt{c} = 2 \\ \texttt{val} &= [a_{11}, a_{12}, a_{21}, a_{22}, a_{13}, a_{33}, a_{34}, a_{43}] \\ \texttt{col_idx} = [1, 2, 2],\ \texttt{row_start} &= [1, 3, 4] \\ \texttt{b_map} &= [15, 1, 7] \end{aligned}$$

E.g., $\begin{bmatrix} a_{33} & a_{34} \\ a_{43} & 0 \end{bmatrix} \implies$ sparsity pattern $\begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix} \implies$ unrolled to 1110

$$\implies 1 \times 2^0 + 1 \times 2^1 + 1 \times 2^2 + 0 \times 2^3 = 7.$$

- **if statements** in SpMV's loop **decreases efficiency**. Two approaches: operate on zeros (Buluç et al., 2011), de Bruijn sequences (Kannan, 2013).

Buluç, A., Williams, S., Oliker, L., & Demmel, J. (2011). Reduced-bandwidth multithreaded algorithms for sparse matrix-vector multiplication. In 2011 IEEE International Parallel & Distributed Processing Symposium (pp. 721-733)

Kannan, R. (2013). Efficient sparse matrix multiple-vector multiplication using a bitmapped format. In 20th Annual International Conference on High Performance Computing (pp. 286-294).

Block Gram-Schmidt procedures

s-step iterative solvers

Matrix power kernels

Homework problems

Practice session